Seg Ter Qua Qui Sex Sab Dom

Aluna: Mábya Rodrigues Vilarinho

Q01. (4) Execução
(2) Compilação
(1) Pré-processamento
(3) Linkagem

Tudo começa no pré-processamento, etapa onde o pré-processador procura a linha (#include <iostream>) que "chama" a biblioteca padrão para que o pré-processador literalmente copie e cole a biblioteca, para que o compilador entenda os comandos. Daí, a próxima é a de compilação, onde o compilador analisa todo o código para identificar erros, dando erro na compilação se, se tudo estiver correto, o arquivo objeto é gerado corretamente.

A terceira etapa é a de linkagem, quando o linker faz a conexão entre os arquivos objetos e as bibliotecas para que o arquivo executável seja criado.

Por último, temos a etapa de execução, quando, se tudo estiver correto, o arquivo executável (aplicativo) será executado pelo processador.

Q02 Não. Isso porque, antes do tudo, o pré-processador processa as diretivas (como o #include), modificando o arquivo original antes da compilação propriamente dita. O que acontece é que, por trás do código-fonte que nós escrevemos existem milhares de linhas de código que são incluídas para que o programa funcione, tornando o arquivo final diferente do que foi escrito inicialmente.

Seg Ter Qua Qui Sex Sab Dom

Q03. O arquivo objeto é o arquivo que foi compilado, sendo que ele passa de código-fonte, linguagem que nós entendemos, para arquivo em linguagem de máquina (binários) que o computador entende. Já o executável final é o arquivo após a linkagem (conexão) entre o arquivo objeto, outros arquivos objeto do projeto e as bibliotecas utilizadas no código. A etapa de linkagem é necessária porque, sem ela, as referências a funções e símbolos externos não seriam resolvidos, impedindo que o executável fosse criado de forma completa.

Q04. No trecho falta o ponto e vírgula (;) ao final da instrução std::cout. Nesse caso, o erro seria detectado na compilação, pois nesta etapa o compilador analisa a sintaxe do código. Como a ausência de ; impede que o compilador compreenda onde a instrução termina, ele interrompe o processo e gera um erro de sintaxe, impedindo que o código seja compilado.

Q05. A diretiva é processada no pré-processamento. Quando essa diretiva é processada, o pré-processador literalmente copia todo o conteúdo do arquivo cabeçalho (iostream) e o cola no seu código-fonte, o que será importante para que o compilador reconheça os comandos de entrada e saída (como cout) na etapa seguinte.

Perfil

Seg Ter Qua Qui Sex Sab Dom

Q06. Ele só chama lá nos arquivos compilados da biblioteca padrão, que já estão em código de máquina. O arquivo de cabeçalho (<iostream>) serve apenas para declarar que a funcionalidade existe e como usá-la, a implementação real é extraída de um arquivo de biblioteca pelo linker durante a fase final de geração do executável.

Q07. a) Falso. O pré-processador apenas executa diretivas (como o #include) antes da compilação real. Sendo assim, ele é como um manipulador de textos, e não entende a sintaxe da linguagem.

b) Verdadeiro. A compilação apenas analisa se a sintaxe e as declarações estão corretas para gerar o arquivo objeto. O erro de linkagem ocorre se o linker não encontrar a implementação real das funções declaradas.

c) Falso. A etapa que transforma o código em código de máquina é a compilação e a linkagem. A execução ocorre apenas quando o usuário abre o arquivo final já pronto para rodar o programa.

Seg Ter Qua Qui Sex Sáb Dom

Q08. Ao compilar apenas o arquivo main.cpp, o linker não consegue encontrar a implementação das funções, pois elas estão em outro arquivo. Para corrigir, é necessário compilar os dois arquivos e linká-los, pois assim o linker poderá conectar os dois arquivos, gerando o executável final.

Q09. # Diretiva do pré-processador: inicia comandos processados antes da compilação.

; Ponto e vírgula: indica ao compilador quando uma instrução acabou.

{} Chaves: delimitam um bloco de código, definindo o escopo de funções, estruturas de controle ou classes.

:: Operador de Resolução de Escopo: identifica e acessa um identificador (como uma variável ou função) dentro de um namespace ou classe específica.

<< Operador de Inserção: é usado principalmente com objetos de fluxo de saída para enviar dados para um dispositivo de exibição ou arquivo.

Q010. A assinatura correta é a B, pois ela não apresenta erros de sintaxe e está escrita completamente em letras minúsculas. Já as outras estão incorretas justamente devido a erros de capitalização. Isso ocorre porque C++ é uma linguagem case-sensitive.

Seg Ter Qua Qui Sex Sab Dom

Q010. A ausência de um único ponto e vírgula causa erros em múltiplas linhas porque o compilador C++ não consegue compreender que o comando chegou ao fim. Ele continua lendo o código como se fosse uma frase infinita.

Ex.: int x = 10 → Ele lerá int x = 10 float y...
float y = 5.0;

Q012. Um token é a menor unidade que o compilador consegue processar individualmente, como se fosse uma "palavra" dentro de uma frase do código.

int resultado = 10 + 5;

→ int é uma palavra-chave (tipo de dado);
→ resultado é um identificador;
→ = é um operador (atribuição);
→ 10 e 5 são literais (valores fixos);
→ + é um operador (aritmético);
→ ; é um delimitador de instrução.

Q013. No contexto de std::cout, ele informa que cout fica dentro do namespace std (outro local). Se escrevêssemos apenas cout, o compilador não encontraria o identificador, pois ele precisa disso próprio ou de uma diretiva específica para buscar a informação dentro da biblioteca.

Perfil CADERNOS

Seg Ter Qua Qui Sex Sab Dom

Q014. Os comentários são completamente ignorados porque servem apenas para anotações que ajudam os programadores a organizarem e entenderem o código. Eles são removidos durante o pré-processamento.
Ex: int x = 5; // x recebe 10.

Q015. Erros léxicos:
→ O Main escrito com M maiúsculo (certo é main)
→ O Return escrito com R maiúsculo (certo é return).

Erros sintáticos:
→ Falta de um : em std::cout.
→ Falta de ; após o return 0.

Q016. A diferença está no local onde o compilador procura os arquivos. Usamos <iostream> para bibliotecas-padrão do C++, onde o compilador busca em pastas do próprio sistema. Já meu arquivo.h é usado para arquivos que o próprio programador criou, fazendo com que o compilador busque primeiro na mesma pasta onde o código principal está salvo.

Q017. Bloco é um conjunto de instruções agrupadas que devem ser executadas juntas. O compilador exige o fechamento para saber exatamente onde termina o escopo da função ou comando. Se esquecer o } da função main, o compilador gerará um erro, pois ele ficará esperando o fechamento do bloco para concluir a compilação.

Seg Ter Qua Qui Sex Sab Dom

Q018. No caso da função da questão, a assinatura seria calcularMedia(int a, int b), que é composta pelo nome da função e sua lista de parâmetros, o tipo de retorno seria float, que indica o tipo de dado do valor que a função deve devolver ao ser encerrada; parâmetros formais seriam int a e int b, que são as variáveis declaradas na definição da função para receber dados; por fim, argumentos seriam valores como 7 e 9, que são os dados reais passados para a função em sua execução.

Q019. A função main é especial porque ela é o ponto de entrada do programa, ou seja, o local onde a execução começa após a compilação. Se existissem duas funções main, o compilador apresentaria um erro de redefinição, pois o sistema não saberia por qual delas iniciar o programa. Por outro lado, se ela não existisse, também ocorreria um erro, pois o compilador não encontraria o ponto de partida necessário para gerar o executável final.

Perfil CADERNOS

Seg Ter Qua Qui Sex Sab Dom

Q020. Inicialmente, o processador executará a função int main e começará a resolver a partir dali. Quando chegar na chamada da função dobrar(5.0f), ele voltará para a definição da função float dobrar, escrita antes da função principal. Ele fará o cálculo, retornará o valor para a main e armazenará esse valor em resultado. Daí, quando chegar em std::cout<<resultado;, ele imprimirá o valor armazenado.

Q021. O escopo de variável é um conjunto de regras que define onde ela é visível dentro de um programa, como por exemplo o escopo local e o escopo global. A variável x declarada dentro de uma função não existe fora dela, porque ela possui escopo local, o que significa que ela é alocada na memória quando a função é chamada e destruída logo após a função ser encerrada. Quando você tenta acessar uma variável fora do seu escopo, o compilador gera um erro, pois ele não consegue encontrar a declaração da variável naquele contexto.

Q022. Declarar é como dar um "aviso" ao compilador de que a função existe, sem dizer o que ela faz. Definir já é escrever o cálculo e o corpo da função de verdade. Usamos o protótipo porque o C++ lê o código de cima para baixo. Se a função estiver lá no fim, o compilador não a vê e dá erro.

Perfil

Seg Ter Qua Qui Sex Sab Dom

O protótipo serve para o compilador "conhecer" a função antes de ela ser usada na main.

→ int somar(int a, int b);

```
int main(){
    int somar =(3,4);
    return 0;
}
int somar (int a, int b){
    return a + b;
}
```

Q023. O return faz duas coisas ao mesmo tempo: ele entrega o resultado final para quem pediu e também encerra a função na hora. Assim que o processador lê o return, ele sai da função e volta para onde estava, ignorando qualquer coisa que esteja escrita abaixo dele.

Exemplo: float teste() { → O que for escrito aqui nunca será lido ou executado.

```
    return 5.0;
}
```

Q024. Como a função foi criada para ser int (inteiro), ela irá ignorar a parte decimal. Sendo assim, se você mandar um 3.7, ele vai devolver apenas o 3. O compilador pode até dar um aviso, mas ele geralmente executa silenciosamente.

Seg Ter Qua Qui Sex Sab Dom

Q025. A diferença é que na passagem por valor a função recebe apenas uma cópia do dado em um novo endereço de memória, enquanto na passagem por referência ela recebe o endereço original da variável.

Q026. int - 4bytes - inteiros - 42
float - 4 bytes - decimais com precisão simples - 3,15F
double - 8bytes - decimais de dupla precisão - 3,53
bool - 1byte - valores lógicos - verdadeiro ou falso.
char - 1 byte - um único caractere - '$'.

Q027. O sufixo é importante pois ele faz com que o número seja tratado como float. Sem f, qualquer literal com ponto decimal é tratado como double por padrão.
Exemplo: float a = 6.2

Q028.

Q030. int x = 10/3; : o resultado será 3, pois int armazena números inteiros.

float y = 10/3; : o resultado será 3.0F, pois a operação será feita como inteiro primeiro.

float z = 10.0F/3; : o resultado será 3.333F, pois float armazena decimais com precisão simples.

Seg Ter Qua Qui Sex Sab Dom

Q031. Significa que, ao declarar uma variável, o computador reserva um endereço físico na memória RAM para armazenar os dados.
→ int resultado = somar(2, 3);
* A CPU reserva um espaço na stack da 'main()' rotulado com o identificador 'resultado'. Daí, o cálculo é realizado e o valor retorno dele é copiado para esse espaço. Quando 'resultado' sair de escopo, o espaço na stack é liberado.

Q033. Namespace é um contêiner lógico de nomes usado para evitar conflitos de nomes. A biblioteca usa std para evitar que funções como cout entre em conflito com outros nomes escritos no código.
→ Acessar cout:
* std::cout << "Olá"; (Acesso direto)
* using std::cout; (Declaração using)
* using namespace std; (Diretiva using).

Q034. A linha do código abaixo funciona como uma esteira de produção: cada << é uma etapa que empurra um item para o fluxo. Isso só funciona porque o operador de inserção retorna a própria referência ao objeto de saída (std::cout).

Perfil CADERNOS

Seg Ter Qua Qui Sex Sab Dom

Q035. std::cout é o objeto de saída padrão, conectado ao terminal, usado para dados normais. já std::cerr é o objeto de saída de erro padrão, usado para mensagens de diagnóstico e erros. O cerr, diferente do cout, não é bufferizado, exibindo a mensagem de erro imediatamente mesmo se o programa travar.

Q036. O std::endl insere uma quebra de linha e força o esvaziamento do buffer (flush); \n apenas insere a quebra de linha. Em alto desempenho, usar \n é melhor, pois evita esvaziamentos contínuos do buffer.

Q037. Torna disponível os objetos de fluxo padrão, os operadores de fluxo, entre outros. Se tentássemos usar std::cout sem incluir <iostream>, receberíamos um erro de compilação, pois o compilador não reconheceria o identificador cout.

Q041. Int: tentar executar n/0 onde n é inteiro causa uma exceção de hardware, e o programa é encerrado imediatamente com um crash.

Float: a divisão não causa crash, mas sim produz valores especiais definidos pelo padrão IEEE 754.

Seg Ter Qua Qui Sex Sab Dom

Q046. Caso A: erro de lógica - a compilação é realizada, mas o resultado é incorreto.
Caso B: erro de semântica - como a variável é int, ela só aceita números inteiros.
Caso C: erro de sintaxe - falta um ; após std::cout << "Olá", gerando um erro de compilação.

Q047. A operação de divisão segue as mesmas regras de uma expressão numérica comum. Sendo assim, o resultado dá errado porque será realizado primeiro b/2 e só depois a soma de a com o resultado. Para corrigi-la teria que escrever entre parênteses: (a+b)/2.0f.

Q048. Uma variável não inicializada é uma variável declarada sem qualquer valor. Declarar apenas int x, faz com que o compilador apenas reserve 4 bytes de espaço na stack, sendo que o valor que estava lá antes permanece.

Q049. 1. Variável não inicializada: usar uma variável antes de atribuir valor a ela;
2. Overflow: exceder o limite do tipo int faz o valor "dar a volta";
3. Conversão implícita: divisão inteira onde se se esperava ponto flutuante, ou truncamento de float para int.

Seg Ter Qua Qui Sex Sab Dom

Q050. • Erro de lógica: int multiplicar(int a, int b){
return a * b;
}

• Erro de sintaxe: float resultado = multiplicar(3, 4);

• Erro de semântica: string em int dá erro.
→ int x = "cinco": erro.

Seg Ter Qua Qui Sex Sab Dom

## Questões - dúvidas

Q028. int a = 3.9; : armazena 3, pois int só armazena inteiros,
float b = 7/2; : armazena 3.0f, pois primeiro é realizada a divisão inteira e só depois ocorre a conversão;
float c = 7.0f / 2; : armazena 3,5f, pois é realizada a divisão em ponto flutuante.

Q029. IEEE 754 representa floats em binário e muitas frações decimais não têm representação exata. A comparação é perigosa, nesse caso porque 0.1f + 0.2f gera resultado diferente de 3.0f, sendo que a comparação == exige igualdade exata bit a bit.

Q032. Int representa valores negativos e positivos, enquanto o unsigned int representa apenas valores ≥0. O perigo é que, como só aceita positivos, ao usar unsigned int o resultado será uma sequência longa de números muito diferente do esperado.

Q038. std::cin é o ostream de entrada padrão (teclado).
Exemplo:

```
int a, b;
std::cin >> a >> b;
std::cout << a+b;
```

Enquanto << insere dados no ostream de saída, >> extrai dados no ostream de entrada.

Q039. Buffer de saída é uma área temporária de memória onde os dados ficam acumulados antes de serem enviados para a tela.
A mensagem não aparece porque, se o programa trava, o buffer nunca é descarregado, o que é diferente std::cerr, que não é bufferizado.

Seg Ter Qua Qui Sex Sab Dom

/ /

40. O operador de igualdade == compara dois valores e retorna true ou false. Por outro lado, o operador de atribuição = armazena um valor em uma variável.

O que acontece é que o compilador avalia o resultado da atribuição (que é 0, considerado false), encerrando a leitura do bloco.

Perfil